

Week 2 • React, part two: forms, lists, and local persistence

Technical Interaction Design • ITU • Autumn 2026

Built 11 September 2026 • commit bc3510e

Latest version: <https://itu-tid.github.io/lecture-notes-pdf/Week-02.pdf>

CONTENTS

Forms and Controlled Components

- Controlled components
- Enter does nothing until the input is inside a form
- The list is what owns the to-dos
- What came up in the lecture

Conditional Rendering

- There are three ways to implement conditional rendering
- `if` and `? :` choose between two things; `&&` only adds one
- `&&` puts a literal 0 on the screen when the left side is a number

The `useEffect` Hook

- A pure function does not reach outside itself, in either direction
- The impure parts belong in an effect, not in the component body
- An empty dependency list means run once, at mount
- Cleanup and the missing dependency list wait until there is a backend

Forms and Controlled Components

Source: [Lectures/Technical/React/Forms-and-Controlled-Components.md](#) — something unclear? [Open an issue](#), or send a pull request.

Until now the app could only add random tasks from a list. Supporting the user to type the task means we have to link React state with HTML input element state. We'll see how to do it below.

Controlled components

An `<input>` normally keeps its own text.

You type, the browser remembers, and React knows nothing about it.

Bridging React with HTML by intercepting keypresses

(more precise is DOM, but we don't care)

We take the job of tracking the state of the input from the browser and give it to React. Two attributes do it – `value` and `onChange` :

```
import { useState } from "react";

function NewTodoForm() {
  const [text, setText] = useState("");

  return (
    <input
      value={text}
      onChange={(e) => setText(e.target.value)}
    />
  );
}
```

- React calls your `onChange` with an [event object](#), named `e` by convention.
- `e.target` is the element that fired it: this input.
- `e.target.value` is the text now in it, *including* the key just pressed.

Read the loop, because it is genuinely circular and that is what makes it feel strange at first:

1. The input shows whatever `text` says.
2. You press a key. The input calls `onChange` , handing it the event.
3. `setText` updates the state, React re-renders, and the input shows the new `text` .

4. Re-rendering does **not** fire `onChange` again. The loop turns once per keystroke, then stops.

What you get for it is **one source of truth**. The input cannot disagree with the app, because there is only one copy of the answer.

Essential Rule: A value you can work out from state should not be state itself

Because `text` is an ordinary value you can read anywhere in the JSX, you can now do things the browser could never do for you:

```
<button disabled={text.length === 0}>Add</button>
```

The button's enabled-ness is *derived* from the text, not stored separately in a second state variable:

```
const [text, setText] = useState("");
const [isDisabled, setIsDisabled] = useState(true); // don't

function handleChange(e) {
  setText(e.target.value);
  setIsDisabled(e.target.value.length === 0); // ...and never forget this
}
```

Now two things have to be kept in step by hand, and one day they will not be. Reach for the derived version whenever you catch yourself about to add a second piece of state that is really about the first.

The pattern has a name: controlled

An input whose value comes from state is called **controlled**. An input left to keep its own text is **uncontrolled**.

You will not write an uncontrolled one in this course. You need the word anyway, because every answer online uses it, and because React puts it in the warning it prints when a `value` arrives late:

"A component is changing an uncontrolled input to be controlled."

That means `value` was `undefined` on the first render, usually because the state started as `undefined` rather than `""`.

Something unclear in this section? Open an issue.

Enter does nothing until the input is inside a form

Give that button an `onClick` and it adds a to-do. Now press **Enter** in the input instead.

Nothing happens — and everybody expects Enter to work.

That is what a `<form>` is for, and it is the reason forms are still worth using in React rather than a naked input and a click handler.

One new thing in the code below: `onAdd`. The form does not own the list of to-dos so it cannot add anything itself. `TodoList` owns it, so it hands the form a function to call when the user submits.

```
function NewTodoForm({ onAdd }) {
  const [text, setText] = useState("");

  function handleSubmit(e) {
    e.preventDefault();
    onAdd(text);
    setText("");
  }

  return (
    <form onSubmit={handleSubmit}>
      <input value={text} onChange={(e) => setText(e.target.value)} />
      <button disabled={text.length === 0}>Add</button>
    </form>
  );
}
```

Convention is that when a function is to be called when a given event happens, the name starts with `on...`

`onChange` and `onSubmit` are React's, spelled exactly so

`onAdd` is a name the parent invents, and it could as well have been `whenSomebodyAdds`. The `on...` spelling is a convention that says *this prop is a function you call when something happens* — nothing more.

One handler catches both the click and the Enter key

`onSubmit` fires for the button click *and* for Enter, so you write the handler once. You also get the semantics for free: a screen reader announces a form, and a phone keyboard offers a **Go** key instead of a plain Return.

preventDefault is what stops the whole page reloading

A form's default behaviour is to send its contents to the server and load whatever comes back — the way the web worked before JavaScript. That would throw away your entire app and reload the page.

We are a single-page application: nothing should ever be sent anywhere or reloaded unless we say so. Take the line out and watch it happen once; the flash of the page reloading is worth seeing.

Every button in a form submits it unless you say otherwise

The Add button above needs no `type`, because every `<button>` already has the type `submit` by default. There are three button types in total:

type	what it does
submit	submits the form — this is the default
button	nothing on its own; it only runs your <code>onClick</code>
reset	empties every field in the form (you rarely want this)

So the Add button is a submit button without saying so, and that is what makes the click and the Enter key arrive at the same handler.

If a button is not meant to submit the form then explicitly make it type `button` - otherwise bugs

The default bites the first time you put a *second* button in a form. A **Clear** or **Cancel** beside Add will submit the form too, and you will spend a while wondering why cancelling adds a to-do. The confusing part is that Clear still *appears* to work: submitting empties the box as well, so the field clears exactly as you intended. Only the list gives it away. Written out, the pair reads clearly:

```
<button type="submit">Add</button>
<button type="button" onClick={() => setText("")}>Clear</button>
```

`type="button"` looks like it says nothing until you know the alternative it is refusing. It means *this is only a button* — do not submit anything.

A controlled input is one you can clear

`setText("")` empties the field, and it is the controlled loop that makes that possible: the text lives in `text`, so clearing `text` clears the box. If the input were holding its own text, you would have to reach into the browser DOM to get at it.

Later, when that `<input>` turns up in more than one place, it is worth pulling into a component of your own — see [Finding the Components](#).

Something unclear in this section? Open an issue.

The list is what owns the to-dos

`onAdd` has been arriving from somewhere all this time. Here is the somewhere.

The form calls `onAdd(text)` with a **string**, because typing a name is all a form knows how to do. Turning that string into a to-do is the list's job:

```
function TodoList() {
  const [todos, setTodos] = useState([]); // each one is { id, text, done }

  function handleAdd(text) {
    setTodos([...todos, { id: crypto.randomUUID(), text, done: false }]);
  }

  function handleRemove(id) {
    setTodos(todos.filter((t) => t.id !== id));
  }

  function handleToggle(id) {
    setTodos(todos.map((t) => (t.id === id ? { ...t, done: !t.done } : t)));
  }

  return (
    <
      <NewTodoForm onAdd={handleAdd} />
      <ul>
        {todos.map((todo) => (
          <TodoItem
            key={todo.id}
            todo={todo}
            onRemove={handleRemove}
            onToggle={handleToggle}
          />
        ))}
      </ul>
    </>
  );
}
```

And the row it renders, which draws one to-do and reports what was done to it:

```
function TodoItem({ todo, onToggle, onRemove }) {
  return (
    <li>
      <input
        type="checkbox"
        checked={todo.done}
        onChange={() => onToggle(todo.id)}
      />
      <span className={todo.done ? "done" : ""}>{todo.text}</span>
      <button onClick={() => onRemove(todo.id)}>x</button>
    </li>
  );
}
```

Three names for one connection

This is the part that trips everyone, and it is worth slowing down on. Look at what `handleAdd` is called in each of the three places it appears:

where	written as	what it is
in <code>TodoList</code>	<code>function handleAdd(text)</code>	the function that actually does the work
at the call site	<code>onAdd={handleAdd}</code>	handing that function to the form
inside <code>NewTodoForm</code>	<code>onAdd</code>	the name the form knows it by

They look like three things. They are one function, named twice — once by the component that owns it, once by the component that will call it.

It is easier if you read it in the form's own voice:

I am a form with an Add button. I know how to collect text and I know when the button was pressed. What I do **not** know is what adding means, because the list is not mine. So: you who want to use me have to hand me a function, and I will call it when Add is pressed. In my own code I will refer to it as `onAdd`, because that is all I know about it.

`TodoList` answers: *here is that function, it is called `handleAdd`, and what it does is put a new to-do on my list.*

That is why the two names differ, and the difference is a convention worth following: **on...** is what the prop is called, **handle...** is what the function is called. `onAdd` describes the *event* from the child's side; `handleAdd` describes the *work* from the parent's side. React's own props follow it —

`onChange` , `onSubmit` , `onClick` — and yours should too, so that a reader can tell at a glance which side of the connection they are looking at.

Why the arrow: `onClick={() => onRemove(todo.id)}`

Look at that line in `TodoItem` and it seems to have an arrow function for no reason. Try removing it and you will see why.

You already know that `onClick={handleRemove}` hands over the function while `onClick={handleRemove()}` calls it immediately, during render, and hands over whatever it returned. So far so good.

But here you need to call it **with a particular to-do's id**, and the moment you write the brackets to pass an argument, you have called it:

```
<button onClick={onRemove(todo.id)}> // calls it while rendering. Every row.  
  Immediately.
```

The arrow is how you get out of that. It is a function you are defining on the spot, which does nothing until it is called:

```
<button onClick={() => onRemove(todo.id)}> // hands over a function that, when run, calls  
  onRemove
```

Read it as “*when clicked, call `onRemove` with this row's id*”. Any time you need a handler that takes an argument, this is the shape.

A checkbox is controlled by `checked` , not by `value`

Everything above about controlled inputs still holds, with one substitution. A text input carries text, so its state prop is `value` . A checkbox carries a yes or a no, so its state prop is `checked` , and the thing to read off the event is `e.target.checked` rather than `e.target.value` .

`TodoItem` above does not need to read the event at all — the row already knows which to-do it is, and `done` is just being flipped — so `onChange={() => onToggle(todo.id)}` ignores its argument entirely.

Use `value` on a checkbox and you get the warning from earlier in this note, “*changing an uncontrolled input to be controlled*”, because `checked` was never given. It is the same warning with a different cause, and the cause is the one you are most likely to hit.

Changing one item in a list means a new object as well as a new array

`handleAdd` builds a new array with `[...todos, x]`. `handleRemove` builds a new array with `filter`. Toggling is the third of these, and the only one that changes an item that is already in the list.

`map` is what does it:

```
todos.map((t) => (t.id === id ? { ...t, done: !t.done } : t))
```

Walk the list. Hand back every to-do unchanged, except the one whose `id` matched — and for that one, hand back **a copy**. Read `{ ...t, done: !t.done }` as *everything this to-do already had*, and then `done` set to the opposite of what it was.

So `map` produces a new array, and the spread produces a new object for the one that changed. Both halves matter, and this line is the shape you will use every time a list item changes.

A to-do is an object now, and that is what makes delete possible

Last week a to-do was a string: `["Buy milk", "Call the landlord"]`. That works right up until a row can be removed, because to delete one you have to say *which* one — and the obvious way to say it is by its text:

```
setTodos(todos.filter((t) => t !== text)); // deletes every "Buy milk"
```

Two people can both put “Buy milk” on the list, and that line removes both of them. The row’s own text cannot identify the row.

The position in the array can, and `filter((t, i) => i !== index)` really does work today. It stops working the moment anything about the list is not a straight click on a rendered row — a reorder, or an update that arrives from a backend while the user is halfway through something. And the position is also what [makes a poor key](#).

So each to-do carries its own name from the moment it is made, an ID made with `crypto.randomUUID()` — built into the browser, no library needed. A counter that goes up by one works just as well.

The id is made once, when the to-do is made, not worked out while rendering. A key has to name the *same* item on every render, and anything computed at render time is a new answer each time. This is the one place in the course where deriving a value instead of storing it is the wrong move.

`TodoItem` has a delete button and cannot delete anything

Look at what `TodoItem` was given: the `todo` to draw, and `onRemove` to call. It does not know what `onRemove` does. It is not deleting anything — it is *reporting a click*, and the list decides what that means.

That is not an accident of this example, it is how the whole tree is wired: data down as props, events back up as functions to call.

What came up in the lecture

Things that happened while this was coded live, rather than things that were planned.

The smallest thing that counts as a form

Asked in class what a `<form>` actually is, and the answer that stuck was by analogy. A slice of bread is not a sandwich. Bread and cheese is a sandwich. In the same way: a button on its own is not a form, an input on its own is not a form, and an input with a button is.

That is not a joke about the definition, it is the definition doing work. A form is the *grouping* — the thing that says these inputs and this button belong together, are submitted together, and are what Enter applies to. Which is exactly why the input had to go inside one before Enter did anything.

“Too many re-renders”

Wiring the checkbox up produced this, immediately:

```
Too many re-renders. React limits the number of renders to prevent an infinite loop.
```

The cause was a missing arrow:

```
<input type="checkbox" onChange={onToggle(todo.id)} />    // wrong
<input type="checkbox" onChange={() => onToggle(todo.id)} /> // right
```

The first one calls `onToggle` *while rendering*. `onToggle` sets state. Setting state causes a render. That render calls `onToggle` again. React counts the loop and stops you.

It is the same mistake as [the arrow above](#), and worth knowing in this louder form too: if you ever see *too many re-renders*, look for a handler you called instead of passed.

JSX syntax is disabled means the file is called .js

Extracting the row into a new file produced an error that says nothing about the filename:

```
JSX syntax is disabled and should be enabled via parser options
```

The file was `TodoItem.js`. Renaming it to `TodoItem.jsx` fixed it.

This is **Vite's rule, not React's** — Vite decides whether to run the JSX transform by looking at the extension, so a `.js` file is treated as ordinary JavaScript and the first `<` is a syntax error. Other React setups are less fussy; the older Create React App compiled JSX in `.js` files quite happily. Worth knowing only because the message points nowhere near the cause.

The point of extracting `TodoItem`

Pulling the row out into its own component is not tidying for its own sake. Left alone, a file like this grows to five hundred lines of JSX, and at that size nobody can read it, nobody can find a bug in it, and every change is a risk.

That is also worth knowing about the AI tools you will be using: they will cheerfully write you five hundred lines of JSX in one component, because you asked for a feature and that is a way to deliver it. The judgement about what should be its own component is still yours, and saying *no, pull that out, I want to be able to read this* is part of using the tool well rather than a nicety on top.

Something unclear in this section? Open an issue.

Exam Questions

1. What does it mean for an input to be *controlled*, and what is the alternative?
2. Why does a form need `e.preventDefault()` in a single-page application?
3. The input shows nothing when you type. Give two different reasons this can happen.
4. Why can a controlled input be cleared after submitting, when an uncontrolled one cannot?
5. You add a Clear button next to Add and now clearing also adds a to-do. Why, and what is the fix?
6. Last week a to-do was a string. Why can the list not stay that way once rows can be deleted?
7. Why is a to-do's `id` created in `handleAdd` rather than worked out while rendering? Everywhere else the course says to derive rather than store.
8. `TodoItem` renders the delete button but cannot delete anything. What does it do instead?

References

- *Adding Interactivity* > [Reacting to Input with State](#)
- *Adding Interactivity* > [Responding to Events](#)

Conditional Rendering

Source: [Lectures/Technical/React/Conditional-Rendering.md](#) — something unclear? [Open an issue](#), or send a pull request.

A component returns JSX. Which JSX it returns can depend on what is in state, or on what it was handed as props.

You need this the moment the app can be in more than one situation, which is sooner than it sounds. A to-do list that can be added to and deleted from can also be **empty**, and an empty `<ul>` (stands for, unordered list) on screen looks like a bug rather than an achievement. So the list needs to say something else when there is nothing in it.

There are three ways to implement conditional rendering

An `if` with an early return

The one you already know from every other language you have written. Note what it guards: the list, and only the list. Put an early return at the top of the component that also renders the form, and you get an app that says *nothing to do* with no way to add anything, for ever:

```
function TodoItems({ todos, onRemove }) {

  if (todos.length === 0) {
    return <p>Nothing to do. Enjoy the afternoon.</p>;
  }

  return (
    <ul>
      {todos.map((todo) => <TodoItem key={todo.id} todo={todo} onRemove={onRemove} />)}
    </ul>
  );
}
```

This works because your react component is a function that returns JSX. Just as you can have early returns in a function, you can have also early returns in a component.

A `<cond>?<true>:<false>` inside the JSX, when the markup around it is shared

Writing that markup out twice would hide how little actually changes:

```
function TodoItems({ todos, onRemove }) {
  return (
    <section>
      <h2>Today</h2>
      {todos.length === 0
        ? <p>Nothing to do. Enjoy the afternoon.</p>
        : <ul>{todos.map((todo) => <TodoItem key={todo.id} todo={todo} onRemove={onRemove} />)}</ul>}
    </section>
  );
}
```

An `&&`, when there is genuinely nothing to show in the other case

```
function TodoItems({ todos, onRemove }) {
  return (
    <section>
      {todos.length === 0 && <p>Nothing to do. Enjoy the afternoon.</p>}
      <ul>{todos.map((todo) => <TodoItem key={todo.id} todo={todo} onRemove={onRemove} />)}</ul>
    </section>
  );
}
```

Something unclear in this section? Open an issue.

`if` and `? :` choose between two things; `&&` only adds one

The first two **choose between two things** — you get the message *or* the list. The third **adds a thing, or does not** — the `<ul>` is always rendered. It is still in the DOM, drawing nothing of its own, which is why the `&&` version is the weakest of the three *for this particular job*: you end up showing the message and an empty list at the same time.

So they are not three styles of one thing. `if` and `? :` answer *which of these two*; `&&` answers *is there anything here at all*. Reach for `&&` when a `? :` would have to end in an awkward `: null`.

A component can also `return null`, which renders nothing at all. The `if` form can be used to hide a component entirely if it returns `null`.

Something unclear in this section? Open an issue.

`&&` puts a literal 0 on the screen when the left side is a number

`&&` returns its **left** side when the left side is falsy — so `{todos.length && <p>...</p>}` renders a literal **0** on the page when the list is empty, because `0` is falsy but is still something React will happily display. Compare explicitly (`=== 0`, `> 0`) and the problem disappears.

Something unclear in this section? [Open an issue](#).

Exam Questions

1. Give the three ways of rendering conditionally, and say when you would reach for each.
2. `{todos.length && <p>Nothing to do</p>}` puts a `0` on the screen. Why, and how do you fix it?
3. Which of the three cannot express an *else*, and what does that tell you about when to use it?

References

- *Describing the UI* > [Conditional Rendering](#)

The useEffect Hook

Source: [Lectures/Technical/React/The-useEffect-Hook.md](#) — something unclear? [Open an issue](#), or send a pull request.

A pure function does not reach outside itself, in either direction

The term comes from functional programming, where the ideal is a **pure function**: one that computes its result and does nothing else. It makes two promises:

1. **It changes nothing outside itself.** Nothing in the world is different afterwards.
2. **It depends on nothing outside itself.** So the same arguments always give the same answer.

One promise points outward, the other inward, and breaking each has its own name.

This one keeps both:

```
function square(i) {  
  return i * i;  
}
```

This one breaks the first. It also writes the answer into **localStorage**, a small key-value store the browser keeps for every site, a handful of megabytes that survives a refresh, and the closest thing to a database you get without a server. It takes strings and gives strings back, and nothing else.

```
function square(i) {  
  localStorage.setItem("square", i * i);  
  return i * i;  
}
```

Writing to storage was not what **square** was for. That is reaching **outward**, and reaching outward has a name: a **side effect**: something the function does besides producing its answer.

Reading from outside breaks the second promise

Writing reached outward. This one leaves the world alone and reaches **inward** instead:

```
function lastSquare() {  
  return localStorage.getItem("square");  
}
```

Nothing is written, so nothing is *affected*, and in the strict sense this is not a side effect at all. But call it twice with the same arguments (there are none) and you can get two different answers, because the answer was never in the arguments. It came from outside.

Which leaves us needing one word that covers both cases, and there is one:

A function is impure when it reaches outside itself in either direction: when it changes something out there, or when its answer depends on something out there.

`square` reaches out, which makes it impure *and* a side effect. `lastSquare` reaches in, which makes it impure and not a side effect at all. Every side effect is impure; not every impure function is a side effect.

React has no use for that distinction. Saving your to-dos and loading them back are the same job done in two directions, and neither belongs in the body of a component. **So from here on this note says *impure*, and means either direction.**

Something unclear in this section? [Open an issue.](#)

The impure parts belong in an effect, not in the component body

A component function has one job: return JSX. Anything else it does — writing to storage, reading it back, changing the page title, calling a backend — makes it impure, and belongs in an effect rather than in the body of the component.

Those four have something in common worth naming: none of them is under React's control. **Anything React does not control is *outside*, and an effect is how you keep your component and the outside in step.**

Leave the `setItem` in the body instead and it runs on *every* render, including the ones that had nothing to do with your to-dos. The effect is where you get to say **when**.

You state the dependency, and React runs the function at the right moment

`useEffect` lets us hook into React and catch the moment a state variable changes, so we can do something about it. Below: a to-do list that writes itself to local storage whenever the list changes.

```
import { useState, useEffect } from "react";

export default function TodoList() {
  const [todos, setTodos] = useState([]); // each one is { id, text, done }

  useEffect(
    () => {localStorage.setItem("todos", JSON.stringify(todos))},
    [todos]
  );

  // ... the form, the list, delete and check-off: unchanged from the form note
}
```

Read it as a sentence: **whenever `todos` changes, write it to local storage.** You never call that function yourself explicitly. You state the dependency, and React runs it at the right moment. You describe the relationship, let React do the work.

`JSON.stringify` is there because local storage only holds **strings**. Hand it an array and you get back `"[object Object]"` on the next load, which is a confusing ten minutes if you have not been warned.

The first argument says what to do, the second says when

First, what to do. A function, usually written in place as an arrow function, though a named one works just as well.

Second, when to do it. An array of the values the effect depends on.

Every effect runs once after the first render, whatever is in the array. After that, React re-runs it whenever one of the dependencies differs from last time. Props count as well as state; anything the effect reads should be in there.

That first run is easy to forget and it is half of most answers: `[todos]` means *once at the start, and again on every change*, not *only on change*.

The array has two special cases.

1. Leave it **empty** and the effect runs once, at mount. That is the next section.
2. Leave it **out altogether** and it runs after every single render, which is almost never what you want and which is why it waits for [useEffect Against a Live Backend](#), where it does real damage.

Updating the page title is the same shape as saving to storage

So that the browser tab says something useful:

```
useEffect(() => {
  document.title = `My To-Dos (${todos.length})`;
}, [todos]);
```

An effect is not for computing something you already have

If a value can be worked out from what you already have, work it out while rendering (`todos.filter(t => !t.done).length`) rather than storing it in state and syncing it with an effect.

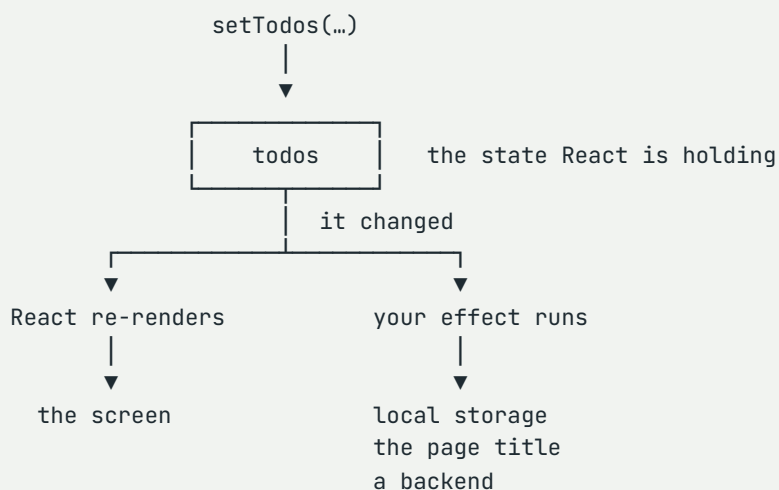
That is a common enough mistake to have a name — **derived state**, kept *in state when it should have been derived*. It makes two sources of truth where one would do, and they drift.

You have met this already: `disabled={text.length === 0}` in [Forms and Controlled Components](#) is the correct way of tracking the enabled state of the button.

An effect is a relationship, not an instruction

You have already met this with `useState`: change the data, and React works out what the screen should look like. See [Reactive Programming](#).

`useEffect` is the same idea pointed outwards. `useState` keeps the *screen* in step with your data; `useEffect` keeps *everything else* in step with it. Same dependency, same automatic re-run, different destination:



You wrote neither arrow. You said what `todos` is, and what depends on it; React does the rest, in both directions.

Something unclear in this section? [Open an issue](#).

An empty dependency list means run once, at mount

An effect with an **empty** array runs once, when the component first appears, and never again.

Mount is React's word for a component appearing for the first time. For its first render, when it goes from not being on screen to being on screen. The opposite is **unmount**, when it is taken off again. A component mounts once and can then re-render any number of times.

An empty dependency list says *nothing to depend on*, so there is never a later change to react to, so it runs at mount and never again.

The list is saved on every change, and read back exactly once

The app saves the list on every change; it needs to read it back exactly once, when it starts. That is the persistence half of the app, complete:

```
function loadTodos() {
  const saved = localStorage.getItem("todos");
  return saved ? JSON.parse(saved) : [];
}

export default function TodoList() {
  const [todos, setTodos] = useState(loadTodos);

  useEffect(() => {
    localStorage.setItem("todos", JSON.stringify(todos));
  }, [todos]);

  // ...
}
```

Note where the load went. `useState(loadTodos)`, the function passed rather than called, asks React to run it once, for the initial value, and never again.

It could have been an effect with an empty array instead, but then the first render would show an empty list and the saved one would appear a moment later, which flickers.

Notice the missing `()`. Not a typo, and you have met the distinction already: on [passing a handler rather than calling it](#): `onClick={handleAdd}` hands over the function, `onClick={handleAdd()}` hands over whatever it returned.

Same thing here, and the consequence is the one you already know: the whole component function runs again on every render: every keystroke, every added to-do. So:

- `useState(loadTodos())`: **you** call it, and you call it on every one of those renders. Storage gets read every time.
- `useState(loadTodos)`: you hand React the function and let it do the calling. It calls it once, for the first render.

Both versions work, which is exactly why this is worth a paragraph: the wasteful one never tells you it is wasteful.

`JSON.parse` is the other half of `JSON.stringify`: storage gave back the string you put in, and this turns it into an array again. The `saved ? ... : []` matters on the very first visit, when there is nothing stored and `getItem` returns `null`.

In two weeks this same shape points at a backend instead of the browser: the same hook, the same dependency array, a Parse query instead of `localStorage`. The swap is smaller than it sounds.

Effects run twice in development, on purpose

React — specifically the `<StrictMode>` wrapper around your app in `main.jsx` — deliberately mounts every component twice while you are developing. In production it happens once.

Which raises the fair question of why anyone would test against something that does not happen. It does happen: **a component mounts every time it appears, and plenty of components appear more than once**. Hide a section and show it again — which you can do already, with [conditional rendering](#) — and it mounts a second time. In a few weeks, when the app has pages, every navigation away and back is another mount.

So the second mount is not an artificial condition. It is a rehearsal, run immediately at your desk, of something a user will do on their own in a fortnight, and a cheap way to find out now that your effect cannot survive it.

So if you see two entries in the console where you expected one, that is why, and it is not a bug you introduced. Saving to local storage twice does no harm: it writes the same thing both times.

It is worth knowing what this is *for*, though. An effect that **sends** something — a message, an order, a payment — would have sent it twice, and that is exactly the bug the double mount is designed to make visible while you are still at your desk.

Something unclear in this section? [Open an issue](#).

Cleanup and the missing dependency list wait until there is a backend

`useEffect` has two further forms: a **cleanup function**, and no dependency list at all. Neither does anything useful against local storage, and both matter once an effect talks to a live backend. They are in [useEffect Against a Live Backend](#), with the problems that make them necessary.

Something unclear in this section? [Open an issue](#).

Exam Questions

1. Explain what this `useEffect` does, and when it runs:

```
useEffect(() => {  
  localStorage.setItem("todos", JSON.stringify(todos));  
}, [todos]);
```

2. What is the difference between these two, and which one saves your work?

```
// Version A  
useEffect(() => {  
  localStorage.setItem("todos", JSON.stringify(todos));  
}, []);  
  
// Version B  
useEffect(() => {  
  localStorage.setItem("todos", JSON.stringify(todos));  
}, [todos]);
```

3. One of these is a side effect and one of them is not. Say which, and why React puts both in an effect anyway.

```
function square(i) {  
  localStorage.setItem("square", i * i);  
  return i * i;  
}  
  
function lastSquare() {  
  return localStorage.getItem("square");  
}
```

4. The to-do list is read back with `useState(loadTodos)` rather than an effect with an empty dependency array. Both work. What does the user see if you use the effect, and why?

5. You expected one line in the console and you got two. What is React doing, why is it harmless for local storage, and what kind of effect would it *not* be harmless for?